Question 1 (10 pts):

Reading (paraphrased):

> *Given a set of transactions, each a set of items, the support of an itemset is the fraction of transactions containing it. Every subset of a frequent itemset is frequent, so candidate k-itemsets need only be formed from frequent (k−1)-itemsets, and any candidate with an infrequent subset can be discarded before its support is counted.*
>
> Reference: Agrawal & Srikant (1994), "Fast Algorithms for Mining Association Rules," VLDB.

Data. B is a binary matrix: one row per cell of the largest Leiden cluster (Project 1, resolution 1.0), one column per gene. B[c, g] = 1 if gene g has at least one count in cell c. The notebook keeps 60 highly variable genes detected in 20–60% of those cells.

Tiny worked example (5 transactions, items A–E, minsup = 3 transactions = 60%):

T1: {A,B,C} T2: {A,B} T3: {A,C} T4: {B,C,D} T5: {A,B,C,E}

Level 1: A=4, B=4, C=4, D=1, E=1 → L_1 = {A},{B},{C}. Items D and E are infrequent, so the seven pairs containing them are never counted.

Level 2 candidates from L_1: {A,B},{A,C},{B,C} (3 of the 10 possible pairs). Counts: AB=3, AC=3, BC=3 → all frequent.

Level 3 candidate {A,B,C} (all three 2-subsets frequent, so it survives pruning). Count: ABC = 2 < 3 → not frequent. Pruning says what cannot be frequent; only counting says what is.

Formulas:

$\text{supp}(X) = |\{T \in D : X \subseteq T\}| \,/\, |D|$

X frequent $\Rightarrow$ every $Y \subseteq X$ is frequent (equivalently: Y infrequent $\Rightarrow$ every $X \supseteq Y$ is infrequent)

Implement:

Complete support (no loop over rows) and my_apriori: level-wise candidate generation by joining frequent (k−1)-itemsets sharing a (k−2)-prefix, pruning by downward closure, then counting survivors. Return every frequent itemset with its support (up to length 3) and the number of candidates counted per level. The assert compares your itemsets and supports with mlxtend's apriori at minsup = 0.30.

Experiment:

Run minsup ∈ {0.25, 0.30, 0.35}. For each, report the number of frequent itemsets of size 1, 2, 3, the number of candidates you counted at levels 2 and 3, and the brute-force numbers C(60,2) and C(60,3). Report the three most frequent gene pairs by name.

Explain:

1. Report, for each minsup, candidates counted at levels 2 and 3 versus the brute-force counts. Which property of support makes this reduction legitimate rather than approximate? State it and explain why it only tells you what cannot be frequent.
2. Compare the number of frequent itemsets across the three thresholds. Is the number of frequent 2-itemsets a property of the data, of the threshold, or of both? Pick one of your top pairs and say what it would take, beyond support, to call it a biological finding.

Question 2 (10 pts):

Reading (paraphrased):

> *An itemset is closed if no proper superset has the same support; it is maximal if no proper superset is frequent. The closed frequent itemsets are a lossless summary: the support of any frequent itemset equals the support of its closure, the closed superset with the largest support. The maximal itemsets are a smaller, lossy summary that preserves which itemsets are frequent but not their supports.*

References: Pasquier, Bastide, Taouil & Lakhal (1999), "Discovering Frequent Closed Itemsets for Association Rules," ICDT; Zaki (2000), "Scalable Algorithms for Association Mining," IEEE TKDE (vertical tidsets).

Tiny worked example (minsup = 3 of 5):

T1: {A,B,C} T2: {A,B,C} T3: {A,B,C,D} T4: {A,D} T5: {B,C}

Frequent (7): A=4, B=4, C=4, AB=3, AC=3, BC=4, ABC=3.

B is not closed: supp(B) = supp(BC) = 4, i.e. every transaction containing B also contains C. Likewise C (→ BC), AB (→ ABC), AC (→ ABC). Closed (3): {A:4, BC:4, ABC:3}. Maximal (1): {ABC}.

Recovery: supp(B) = support of its closure BC = 4 ✓; supp(AB) = support of ABC = 3 ✓. From the maximal set alone you know A, B, C, AB, AC, BC are frequent but not whether A has support 4 or 3.

Vertical representation: tidset(A) = {1,2,3,4}, tidset(B) = {1,2,3,5}; supp(AB) = |{1,2,3,4} ∩ {1,2,3,5}| / 5 = 3/5. Support becomes set intersection; no scan, no tree.

Formulas:

$\text{closed}(X) \Leftrightarrow \nexists\, Y \supset X$ with $\text{supp}(Y) = \text{supp}(X)$ $\quad$ $\text{maximal}(X) \Leftrightarrow \nexists$ frequent $Y \supset X$

$\text{supp}(X) = \max \{ \text{supp}(Y) : Y \text{ closed}, Y \supseteq X \}$ $\quad$ $\text{supp}(X) = | \cap_{i \in X} \text{tidset}(i) | \,/\, |D|$

Implement:

Complete closed_itemsets, maximal_itemsets, recover_support and tidset_support. Asserts: every frequent support is recovered exactly from the closed set; maximal ⊆ closed ⊆ frequent; tidset supports equal Apriori supports for all frequent pairs.

Experiment:

The assert first checks your functions on the DB-2 toy above (closed = {A, BC, ABC}, maximal = {ABC}, recovery of B and AB). Then, at minsup ∈ {0.25, 0.30, 0.35} on the real matrix, report the counts of frequent, exactly-closed, approximately-closed (no superset within δ = 1% and δ = 5% of cells) and maximal itemsets, with the maximal/frequent ratio.

Explain:

1. Report the counts. On DB-2 the closed set compresses 7 → 3 losslessly; on the real matrix, how much does exact closure compress, and why? Use recover_support to explain how a support is reconstructed from the closed set, and state exactly what the maximal set loses.
2. In single-cell data a gene can be expressed but still read as 0 ("dropout"), so detection is a noisy indicator of expression. Compare the exact closed count with the δ = 1% and δ = 5% counts. Why is exact closure essentially absent in single-cell detection data even when two genes are biologically co-regulated (think dropout)? Interpret supp(G) ≈ supp(G ∪ H) biologically, and state precisely what the lossless guarantee becomes once a tolerance δ is allowed.

Question 3 (10 pts):

Reading (paraphrased):

> *A rule $X \rightarrow Y$ has confidence $supp(X \cup Y)/supp(X)$, the conditional frequency of Y given X. Confidence ignores the baseline frequency of Y: when Y is common, a high-confidence rule may describe a negative association. Lift divides confidence by supp(Y) and equals 1 under independence. Associations measured on pooled data can reverse sign within every subgroup when group membership is a confounder.*

References: Brin, Motwani & Silverstein (1997), "Beyond Market Baskets: Generalizing Association Rules to Correlations," SIGMOD; Simpson (1951), J. Royal Statistical Society B.

Tiny worked example (1,000 customers):

coffee 800, tea 200, both 150. conf(tea → coffee) = 150/200 = 0.75. Baseline P(coffee) = 0.80. lift = 0.75/0.80 = 0.94 < 1: tea drinkers buy coffee less often than everyone else. The rule is true; the promotion it suggests is wrong.

Simpson. Rule X → Y across two groups:

students : X 200 -> Y 150 (75%) not-X 800 -> Y 560 (70%)

adults : X 800 -> Y 80 (10%) not-X 200 -> Y 10 (5%)

pooled : X 1000 -> Y 230 (23%) not-X 1000 -> Y 570 (57%)

X raises Y inside each group and lowers Y in the pooled table; the group is the latent variable.

Formulas:

$conf(X \rightarrow Y) = supp(X \cup Y) / supp(X)$ $lift(X \rightarrow Y) = conf(X \rightarrow Y) / supp(Y) = P(X,Y) / (P(X) \cdot P(Y))$

$leverage(X \rightarrow Y) = supp(X \cup Y) - supp(X) \cdot supp(Y)$

Implement:

Complete my_rules: from every frequent itemset of size ≥ 2 generate all rules X → Y with X, Y non-empty and disjoint, with support, confidence, lift and leverage. The assert compares rule count, confidence and lift with mlxtend's association_rules on the same frequent itemsets.

Experiment:

(a) Report how many rules appear in both the top-10 by confidence and the top-10 by lift; report how many rules have lift < 1 and print the five of them with the highest confidence together with their consequent's baseline support. (b) Pool the cells of the two largest Leiden clusters. For every gene pair compute lift on the pooled cells and inside each cluster; count pairs with pooled lift > 1.1 and lift < 1 inside both clusters, and print the strongest example with its three values.

Explain:

1. Report the overlap and the number of rules with lift < 1. Take the highest-confidence rule among them and explain, using its confidence and its consequent's baseline support, why a confidence that sounds respectable can describe a negative association. Which measure would you report to a biologist, and why?
2. Each Leiden cluster is treated as one cell population (approximately one cell type). Report the number of reversals and your example pair with its three lift values. What is the latent variable, and at which level is the association "real" for describing one cluster? Explain why pooling two populations manufactures co-occurrence.

Question 4 (10 pts):

Reading (paraphrased):

> *A sequence database stores ordered lists of items. A pattern ⟨a_1, …, a_k⟩ is contained in a sequence if its items occur in that order, not necessarily contiguously. Support is the fraction of sequences containing the pattern. The apriori property holds: every subsequence of a frequent sequential pattern is frequent, so patterns can be grown level by level, or by projecting the database on a frequent prefix.*

References: Srikant & Agrawal (1996), "Mining Sequential Patterns: Generalizations and Performance Improvements," EDBT (GSP); Pei et al. (2001), "PrefixSpan," ICDE.

Data. 500 synthetic browsing sessions over eight page types A–H, generated in the notebook. In 35% of sessions the ordered pattern A … D … G is planted at random positions.

Tiny worked example (4 sequences):

S1 = <a,b,c> S2 = <a,c,b> S3 = <b,a,c> S4 = <a,b>

⟨a,b⟩: S1 ✓, S2 ✓ (a before b with a gap), S3 ✗ (b precedes a), S4 ✓ → supp = 3/4. ⟨b,a⟩: only S3 → 1/4. ⟨a,c⟩: S1, S2, S3 → 3/4. ⟨a,b,c⟩: only S1 → 1/4. The unordered itemset {a,b} is contained in all four sequences (4/4): the set representation cannot see that a usually comes first.

Formulas:

$\langle a_1 \ldots a_k \rangle \sqsubseteq S \iff \exists\, i_1 < \ldots < i_k$ with $S[i_j] = a_j$ $\text{supp}(p) = |\{S : p \sqsubseteq S\}| \,/\, |DB|$

Implement:

Complete contains_subsequence (one pass with a pointer), seq_support, and mine_sequences: level-wise growth that extends each frequent k-pattern by one frequent item and prunes candidates having an infrequent k-subsequence. Asserts use the 4-sequence example above.

Experiment:

Mine the session database at minsup = 0.15 up to length 3; report the top-10 patterns. Report the sequence support of ⟨A,D,G⟩, ⟨G,D,A⟩ and ⟨D,A,G⟩ alongside the itemset support of {A,D,G}.

Explain:

1. Report the three sequence supports and the itemset support. What information does the itemset representation discard, and how does your experiment show that this is where the planted signal lives?
2. State the apriori property precisely for ordered patterns (which sub-patterns must be frequent?) and explain one way in which candidate generation for sequences is more expensive than for itemsets of the same length.

Question 5 (10 pts):

Reading (paraphrased):

> *Mining results can be assessed against a null model that preserves the row and column margins of the binary data (each cell's number of detected genes, each gene's detection count) but randomizes which genes co-occur. Repeated checkerboard swaps generate such matrices; a statistic's empirical p-value is the fraction of null matrices on which it is at least as extreme. When thousands of patterns are tested, the per-pattern significance level must be corrected or many spurious "discoveries" are guaranteed.*

References: Gionis, Mannila, Mielikäinen & Tsaparas (2007), "Assessing Data Mining Results via Swap Randomization," ACM TKDD; Webb (2007), "Discovering Significant Patterns," Machine Learning.

Tiny worked example:

rows r1, r2 ; columns c1, c2 : [1 0] swap [0 1]
[0 1] ------> [1 0]

Margins unchanged, co-occurrence changed. Empirical p = (1 + #{null ≥ observed}) / (B + 1); observed 5 against nulls {1, 2, 3, 5, 7} gives 3/6 = 0.5.

Multiple comparisons: 60 genes give 1,770 pairs; testing each at $\alpha = 0.05$ yields ≈ 88 "significant" pairs under a true null. Bonferroni uses $\alpha/1{,}770 \approx 2.8\times10^{-5}$, but an empirical p from B null matrices is never below 1/(B+1): 40 matrices give a floor of 0.024, so no pair can be certified at that level without an approximation.

Formulas:

swap: $B[r_1,c_1]=B[r_2,c_2]=1$, $B[r_1,c_2]=B[r_2,c_1]=0$ → flip all four p = (1 + #{null ≥ obs}) / (B + 1)

Implement:

Complete swap_randomize (sample two 1-entries, swap only when the checkerboard condition holds) and empirical_p. Asserts: row and column sums unchanged; at least 5% of ones moved; the hand-computed p.

Experiment:

Generate 40 null matrices (5 × nnz attempted swaps each). Report the observed number of gene pairs with support ≥ 0.30, the null mean/min/max and the empirical p. For each pair compare its observed lift with the 95th percentile of its null lifts; report how many exceed it versus the ≈ 88 expected by chance. Report the per-pair empirical-p floor 1/41, how many pairs sit at it, the number of null matrices Bonferroni would require, and the count from the normal-approximation shortcut z = (lift − null mean)/null SD > z_Bonferroni, together with the null-lift histogram of the top pair.

Explain:

1. Report the observed count, the null range and the p-value. What does swap randomization hold fixed, and therefore what null hypothesis does this p test? Name one kind of structure that would survive this null and therefore not be "discovered" by it.
2. Report the pairs above the per-pair 95th percentile versus ≈ 88 expected by chance, how many sit at the empirical-p floor, and the normal-approximation Bonferroni count. Why can 40 randomizations never certify any pair at the Bonferroni level, how many would be needed, and

what assumption does the z-approximation buy you in exchange? Why does "1,770 pairs tested" change what a single pair's lift means?

Question 6 (10 pts):

Reading (paraphrased):

> *Pointwise mutual information compares how often two items co-occur with how often they would co-occur if independent. Positive PMI (negatives clipped to zero) removes the dominance of ubiquitous items in raw co-occurrence counts. The skip-gram word-embedding objective is implicitly factorizing a shifted PMI matrix, so a low-rank factorization of the PPMI matrix yields item embeddings whose cosine similarity reflects co-occurrence patterns.*

References: Church & Hanks (1990), "Word Association Norms, Mutual Information, and Lexicography," Computational Linguistics; Levy & Goldberg (2014), "Neural Word Embedding as Implicit Matrix Factorization," NeurIPS.

Data. Co-detection counts over all 2,700 cells for ≈300 genes (drawn from ALL genes, not only HVGs) detected in 5–80% of cells, plus the known marker genes CD3D, MS4A1, LYZ, NKG7 and their partners whenever they are present in the dataset. C[i,j] = number of cells detecting both genes; C[i,i] = detection count of gene i; N = number of cells. T cell: CD3D, CD3E, IL7R · B cell: MS4A1, CD79A, CD79B · Monocyte: LYZ, S100A8, S100A9, CST3, FCGR3A · NK: NKG7, GNLY, GZMB

Tiny worked example (N = 10 cells):

gene i detected in 5 cells, gene j in 4, both in 4: PMI = ln(4·10 / (5·4)) = ln 2 = 0.693 → PPMI 0.693. gene k detected in 6, i and k together in 3: PMI = ln(3·10 / (5·6)) = ln 1 = 0 → PPMI 0 (exactly what independence predicts).

Formulas:

$PMI(i,j) = \ln(C_{ij} \cdot N / (C_{ii} \cdot C_{jj}))$ $PPMI = \max(PMI, 0)$, $PPMI = 0$ where $C_{ij} = 0$

$M = PPMI \approx U \Sigma V^T$ (rank 20) $E = U \Sigma^{1/2}$ (one row per gene)

Implement:

Complete ppmi (vectorized). The assert checks the worked example. The notebook builds C, factorizes PPMI with a rank-20 SVD and prints, for CD3D, MS4A1, LYZ and NKG7, the five nearest genes by cosine similarity under raw co-detection counts and under the PPMI embedding.

Experiment:

Report both neighbor lists for the four query genes and the most-detected gene in the vocabulary. Count how often that hub gene appears in the raw-count lists versus the PPMI lists.

Explain:

1. Which list recovers the partners listed in the Data section (T-cell, B-cell, monocyte and NK markers) and which is dominated by the hub gene? Explain mechanically why PMI's denominator removes the hub.
2. Levy & Goldberg show that skip-gram implicitly factorizes a shifted PMI matrix. In this pipeline, what plays the role of word, context and corpus? Name one assumption about which genes are "similar" that this embedding encodes and that a biologist might reject.

Question 7 (10 pts):

Reading (paraphrased):

> *When items arrive faster than they can be stored, exact counting of every distinct item needs linear memory. Misra–Gries keeps k counters: every estimate is at most N/(k+1) below the truth and never above it. Count-Min keeps a $d \times w$ table indexed by d hash functions: its estimate never falls below the truth and, with $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$, exceeds it by more than εN with probability at most δ.*

References: Misra & Gries (1982), "Finding Repeated Elements," Science of Computer Programming; Cormode & Muthukrishnan (2005), "An Improved Data Stream Summary: The Count-Min Sketch and its Applications," J. Algorithms.

Data. (a) A synthetic Zipf(1.5) stream of ≈200,000 tokens over ≤ 5,000 items. (b) The real stream of (cell, detected gene) tokens: each detected HVG in each cell is one arrival; the item is the gene.

Tiny worked example (Misra–Gries, k = 2 counters):

stream: a b a c a b a (N = 7; true a=4, b=2, c=1)

a:{a:1} b:{a:1,b:1} a:{a:2,b:1} c: no slot -> decrement all -> {a:1}

a:{a:2} b:{a:2,b:1} a:{a:3,b:1} final estimates a=3, b=1, c=0

Every error (a: 1, b: 1, c: 1) is ≤ N/(k+1) = 7/3 ≈ 2.33, and no estimate exceeds the truth.

Formulas:

Misra–Gries: $\hat{f}(x) \le f(x)$ and $f(x) - \hat{f}(x) \le N / (k+1)$

Count-Min: $\hat{f}(x) = \min_r T[r, h_r(x)] \ge f(x)$; $P[\, \hat{f}(x) > f(x) + \varepsilon N \,] \le \delta$ for $w = \lceil e/\varepsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$

Implement:

Complete misra_gries and the add / estimate methods of CountMin using the supplied hash. Asserts: Misra–Gries never over-estimates and its error respects N/(k+1); Count-Min never under-estimates.

Experiment:

On the Zipf stream: for Count-Min with d = 4 and $w \in \{50, 200, 1000\}$, report max and mean overestimate, the bound εN with ε = e/w, and the fraction of items exceeding it. For Misra–Gries with $k \in \{20, 50, 200\}$, report counters kept, max underestimate, N/(k+1), and how many of the true top-10 are recovered. On the real gene stream, report N, the share of the stream carried by the most frequent gene, the Misra–Gries guarantee N/(k+1) and the number of genes above it for $k \in \{100, 500, 2000\}$ with the top-5 estimates, and the Count-Min (w = 500) estimates for the top-5 genes.

Explain:

1. Compare max overestimate against εN at the three widths and report the fraction exceeding it. Which side of the truth does each sketch err on, and why does that follow from "min over rows" versus "decrement all counters"? When would you prefer each error direction?
2. On the Zipf stream, how many of the true top-10 does Misra–Gries recover at k = 20, 50, 200? On the real gene stream, why does k = 100 return 0 for every top gene while k = 2000 does

not? Use the printed N/(k+1) thresholds and the most frequent gene's share of the stream to say what "frequent" can even mean when the mass is spread over thousands of items.

Question 8 (10 pts):

Reading (paraphrased):

> *A Bloom filter stores a set in m bits using k hash functions: insertion sets k bits; a query answers "possibly present" if all k bits are set. It never reports a false negative; its false-positive rate after n insertions is approximately (1 − e^{−kn/m})^k. Reservoir sampling maintains a uniform random sample of fixed size s from a stream of unknown length: keep the first s items, then replace a uniformly chosen slot with item t with probability s/t.*

References: Bloom (1970), "Space/Time Trade-offs in Hash Coding with Allowable Errors," CACM; Vitter (1985), "Random Sampling with a Reservoir," ACM TOMS.

Data. Tokens of the form "cell:gene" for detected genes: 20,000 from the largest cluster are inserted; 20,000 from the second-largest cluster that were not inserted are queried. For sampling, the stream of all 2,700 cells.

Tiny worked example:

Bloom with m = 100 bits, k = 3, after n = 10 insertions: FPR ≈ (1 − e^{−30/100})^3 = (0.259)^3 ≈ 0.017. The same filter can never say "absent" for an inserted item.

Reservoir with s = 1 over a stream of 3 items: item 1 is kept initially; it survives item 2 with probability 1/2 and item 3 with probability 2/3, so P(item 1 ends in the sample) = 1·(1/2)·(2/3) = 1/3 = s/n, the same as every other item.

Formulas:

FPR ≈ (1 − e^{−kn/m})^k P[item t replaces a slot] = s/t (t > s) P[any item in final sample] = s/n

Implement:

Complete Bloom.add, Bloom.maybe_contains and reservoir_sample (Algorithm R). Asserts: no false negatives; measured FPR within 0.02 of theory on a synthetic set; reservoir size and uniqueness.

Experiment:

(a) For m ∈ {50k, 200k, 800k} bits and k ∈ {1, 3, 7}, report bits per item, measured FPR and the theoretical value; compare the bytes of the exact string set with the filter's packed size (the provided class stores the m bits in a bytearray, so its memory is m/8 bytes). (b) Draw 300 reservoirs of 100 cells; report the mean, min and max inclusion frequency against s/n and the mean and SD of the 300 estimates of mean mitochondrial percentage against the full-data value, with the histogram.

Explain:

1. Report measured versus theoretical FPR for the nine settings. For fixed m, why does increasing k first lower and then raise the FPR? State the two error types a Bloom filter can and cannot make, and why that asymmetry suits the duplicate-screening task of Project 1, Q4.

2. Using the inclusion frequencies and the spread of the 300 estimates, argue that Algorithm R yields a uniform sample without knowing n in advance. Why must the replacement probability be exactly s/t at step t, and what would go wrong with a fixed probability?

Question 9 (10 pts):

Reading (paraphrased):

> *DGIM counts the 1s among the last N bits in $O(\log^2 N)$ memory: 1s are grouped into buckets of power-of-two sizes, at most two per size, merging the two oldest when a third appears. The estimate is the size of every in-window bucket except the oldest plus half the oldest; relative error is at most 50%. Drift is detected by comparing a reference window with the current window and flagging a mean difference above a Hoeffding-type bound.*

References: Datar, Gionis, Indyk & Motwani (2002), "Maintaining Stream Statistics over Sliding Windows," SIAM J. Computing; Bifet & Gavaldà (2007), "Learning from Time-Changing Data with Adaptive Windowing," SDM (ADWIN).

Data. A time-ordered stream of 1,500 cells: 900 from the largest cluster, then a 50/50 mixture of the two largest. The bit at each step is whether the cell detects the gene whose detection rate differs most between the two clusters.

Tiny worked example (DGIM):

bits (t = 1..10): 1 1 0 1 1 1 0 1 1 1

After t = 10 the buckets are (end-time, size): (5, 4), (8, 2), (9, 1), (10, 1). Query the last N = 6 bits (t = 5..10): buckets ending after t = 4 are all four; estimate = 1 + 1 + 2 + ½·4 = 6. Exact count = 5. Error 1 ≤ 50% of 5.

Drift bound with $\delta = 0.05$ and two windows of 300: $m = 1/(1/300 + 1/300) = 150$, $\varepsilon_{cut} = \sqrt{(\ln(80)/300)} \approx 0.121$, so a change in the detection fraction larger than ≈ 0.12 is flagged.

Formulas:

DGIM estimate(N) = $\Sigma_{\text{in-window buckets except oldest}}$ size + ½ · size(oldest) buckets = O(log N) of O(log N) bits

$\varepsilon_{cut} = \sqrt{\ln(4/\delta) / (2m)}$, $m = 1 / (1/n_{ref} + 1/n_{cur})$; flag when $|mean_{ref} - mean_{cur}| > \varepsilon_{cut}$

Implement:

Complete DGIM.add (append a size-1 bucket, merge the two oldest of any size that appears three times, cascading), DGIM.query, and drift_detector (fixed reference window, sliding current window). Asserts: the worked example returns 6; relative error ≤ 50% along a synthetic Bernoulli stream; the detector fires on a step and stays silent on a constant stream.

Experiment:

Plot the exact count of marker-positive cells in the last 300 versus the DGIM estimate along the stream, marking the change point; report the worst relative error and the number of buckets stored at the end. Report the time at which drift is flagged, the delay from the true change point, and whether a false alarm occurs on the pre-change segment.

Explain:

1. Report the worst DGIM relative error and the buckets stored versus the 300 bits an exact window keeps. Explain where the error comes from (which bucket, and why half) and why memory is logarithmic in the window length.
2. Report detection time and delay. Which two quantities set the detector's sensitivity, and how would a smaller δ or a smaller n_cur trade false alarms against delay? On a stream, what second question joins Week 6's "normal according to whom"?

Question 10 (10 pts):

Reading (instructor-written scenario):

A short report built from this notebook — frequent gene sets with lift, a swap-randomized null, a PPMI gene embedding, and streaming summaries of the cell–gene token stream — has been submitted for review. Respond to each comment below using only results already in your notebook.

Comment 1: "Frequent is not interesting."

"The authors report dozens of frequent gene sets, but any pair of broadly expressed genes is frequent. Nothing here distinguishes co-regulation from co-abundance."

Respond in one or two paragraphs with evidence from Q3 (confidence vs lift, Simpson) and Q5 (swap-randomized null, multiple comparisons): which of your frequent pairs survive, and what exactly did each test rule out?

Comment 2: "Why approximate?"

"The dataset has 2,700 cells. Sketches, Bloom filters and reservoirs are unnecessary; just count."

Respond using Q7, Q8 and Q9. For each structure, state the exact-counting cost it replaces, the error guarantee you measured, and one regime (data size or arrival pattern) where the reviewer is right and one where they are wrong.

Comment 3: "Convince me."

"What single additional analysis would most strengthen the paper?"

Propose one experiment in 3–5 sentences: what you would run, what result you predict, and what result would instead make you change the pipeline. An experiment that cannot fail is not an experiment.